

TIME COMPLEXITY

Type 2: Multiplicative Loops and Logarithmic Complexity

Contents

1	Introduction	3
2	Important Logarithmic Equations	3
2.1	Equation 1: Basic Logarithm	3
2.2	Equation 2: General Logarithm	4
2.3	Equation 3: Change of Base	4
3	Basic Idea of a Multiplicative Loop	5
4	Important Difference from Linear Loops	5
5	Case 1: Starting from 0	6
5.1	Time Complexity	7
5.2	Corrected Version	7
6	Case 2: Starting from 1	8
6.1	Step 1: Find the values	8
6.2	Step 2: Apply the loop condition	8
6.3	Step 3: Apply logarithm	8
6.4	Step 4: Big-O	8
7	Case 3: Starting from 2	9
7.1	Step 1: Find the values	9
7.2	Step 2: Apply the condition	9
7.3	Step 3: Big-O	9
8	Case 4: Starting from -1	10
9	Summary of Starting Values	10
10	General Formula for Multiplicative Loops	11
11	Example: Multiplication by 3	11
12	Example: Multiplication by 4	12
13	Why Is It Called Logarithmic?	13

14 Easy Method to Solve Multiplicative Loops	13
15 Important Special Cases	14
16 Important Warning	14
17 Quick Revision	15
18 Student Shortcut	15
19 Final Summary	16

1 Introduction

In the previous type of loop, the value of the loop variable was changed by addition or subtraction.

For example:

```
for(int i = 0; i < n; i++)  
{  
    // statements  
}
```

Here the value changes as:

$$0, 1, 2, 3, 4, \dots, n - 1$$

The loop variable increases by a constant amount.

Therefore, the number of iterations is proportional to n .

Hence:

$$O(n)$$

Now consider a different type of loop:

```
for(int i = 1; i < n; i = i * 2)  
{  
    // statements  
}
```

Here the values are:

$$1, 2, 4, 8, 16, 32, \dots$$

The value is multiplied by 2 in every iteration.

Such loops have **logarithmic time complexity**.

$$O(\log n)$$

2 Important Logarithmic Equations

Before analysing logarithmic loops, remember the following equations.

2.1 Equation 1: Basic Logarithm

If

$$2^k = n$$

then

$$k = \log_2 n$$

This means that if a value doubles k times and reaches n , then the number of multiplications is approximately:

$$\log_2 n$$

2.2 Equation 2: General Logarithm

If

$$a^k = n$$

then

$$k = \log_a n$$

For example:

$$3^k = n$$

gives

$$k = \log_3 n$$

2.3 Equation 3: Change of Base

In computer science, the base of the logarithm is often omitted.

For constant bases:

$$\log_2 n, \quad \log_3 n, \quad \log_{10} n$$

are all considered:

$$O(\log n)$$

because changing the base only changes the constant factor.

The change-of-base formula is:

$$\log_a n = \frac{\log_b n}{\log_b a}$$

Therefore:

$$\log_2 n = \frac{\log n}{\log 2}$$

Since $\frac{1}{\log 2}$ is a constant,

$$O(\log_2 n) = O(\log n)$$

3 Basic Idea of a Multiplicative Loop

Consider:

```
for(int i = 1; i < n; i = i * 2)
{
    // statement
}
```

Let us observe the values of i .

Iteration	i
1	1
2	2
3	4
4	8
5	16
6	32
7	64
$\vdots$	$\vdots$

After k iterations:

$$i = 2^k$$

The loop continues while:

$$i < n$$

Therefore:

$$2^k < n$$

Taking logarithm on both sides:

$$k < \log_2 n$$

Hence the number of iterations is proportional to:

$$\log_2 n$$

Therefore:

$$O(\log n)$$

4 Important Difference from Linear Loops

For a linear loop:

```
for(int i = 0; i < n; i++)
```

the values are:

$$0, 1, 2, 3, 4, \dots, n - 1$$

The value increases slowly.

Number of iterations:

$$n$$

Therefore:

$$O(n)$$

For a multiplicative loop:

```
for(int i = 1; i < n; i = i * 2)
```

the values are:

$$1, 2, 4, 8, 16, \dots$$

The value increases very quickly.

Number of iterations:

$$\log_2 n$$

Therefore:

$$O(\log n)$$

Loop Type	Change in i	Time Complexity
Linear	$i = i + 1$	$O(n)$
Linear	$i = i + 5$	$O(n)$
Multiplicative	$i = i * 2$	$O(\log n)$
Multiplicative	$i = i * 3$	$O(\log n)$

5 Case 1: Starting from 0

Consider:

```
for(int i = 0; i < n; i = i * 2)
{
    // statement
}
```

At first glance, this looks like a logarithmic loop.

However, there is an important problem.

Initially:

$$i = 0$$

After multiplication:

$$i = i * 2$$

Therefore:

$$i = 0 * 2 = 0$$

The value remains zero forever:

$$0, 0, 0, 0, 0, \dots$$

Therefore, the loop never terminates.

5.1 Time Complexity

This loop does not finish for $n > 0$.

Therefore, it is not correct to write:

$$O(\log n)$$

The correct observation is:

Infinite loop so Time Complexity is Not Defined

5.2 Corrected Version

If we want logarithmic complexity, start from 1:

```
for(int i = 1; i < n; i = i * 2)
{
    // statement
}
```

Now:

$$1, 2, 4, 8, 16, \dots$$

and the time complexity is:

$$O(\log n)$$

Important:

$$0 \times 2 = 0$$

Therefore, a multiplicative loop should generally not start from zero.

6 Case 2: Starting from 1

Consider:

```
for(int i = 1; i < n; i = i * 2)
{
    // statement
}
```

6.1 Step 1: Find the values

The values of i are:

$$1, 2, 4, 8, 16, 32, \dots$$

After k iterations:

$$i = 2^k$$

6.2 Step 2: Apply the loop condition

The loop continues while:

$$i < n$$

Therefore:

$$2^k < n$$

6.3 Step 3: Apply logarithm

Taking $\log_2$ on both sides:

$$k < \log_2 n$$

Therefore, the number of iterations is:

$$\log_2 n$$

6.4 Step 4: Big-O

$$O(\log n)$$

Therefore:

$$\text{for(int i=1; i<n; i=i*2)} \Rightarrow O(\log n)$$

7 Case 3: Starting from 2

Consider:

```
for(int i = 2; i < n; i = i * 2)
{
    // statement
}
```

7.1 Step 1: Find the values

The values are:

$$2, 4, 8, 16, 32, \dots$$

After k iterations:

$$i = 2 \cdot 2^k$$

or equivalently,

$$i = 2^{k+1}$$

7.2 Step 2: Apply the condition

The loop continues while:

$$2^{k+1} < n$$

Taking logarithm:

$$k + 1 < \log_2 n$$

Therefore:

$$k < \log_2 n - 1$$

The number of iterations is approximately:

$$\log_2 n - 1$$

7.3 Step 3: Big-O

The constant -1 is ignored in Big-O notation.

Therefore:

$$O(\log n)$$

Hence:

$$\text{for(int i=2; i<n; i=i*2)} \Rightarrow O(\log n)$$

8 Case 4: Starting from -1

Consider:

```
for(int i = -1; i < n; i = i * 2)
{
    // statement
}
```

Now observe the values:

$$-1, -2, -4, -8, -16, -32, \dots$$

The value becomes more and more negative.

For example:

$$-1 \times 2 = -2$$

$$-2 \times 2 = -4$$

$$-4 \times 2 = -8$$

Therefore, i will never become greater than or equal to n when n is a positive integer.

The loop continues indefinitely.

Hence:

Infinite loop so Time Complexity is Not Defined

It is **not**:

$$O(\log n)$$

Important:

Multiplication by 2 gives logarithmic complexity only when the loop moves toward the terminating condition.

9 Summary of Starting Values

Consider the general loop:

```
for(int i = START; i < n; i = i * 2)
{
    // statement
}
```

Different starting values give different behaviour.

Starting Value	Values of i	Result
0	0, 0, 0, 0, ...	Infinite loop
1	1, 2, 4, 8, ...	$O(\log n)$
2	2, 4, 8, 16, ...	$O(\log n)$
-1	-1, -2, -4, -8, ...	Infinite loop

10 General Formula for Multiplicative Loops

Consider:

```
for(int i = c; i < n; i = i * d)
{
    // statement
}
```

where $c > 0$ and $d > 1$.

The values of i are:

$$c, cd, cd^2, cd^3, \dots$$

After k iterations:

$$i = cd^k$$

The loop terminates when:

$$cd^k \geq n$$

Therefore:

$$d^k \geq \frac{n}{c}$$

Taking logarithm:

$$k \geq \log_d \left(\frac{n}{c} \right)$$

Hence the number of iterations is approximately:

$$\log_d \left(\frac{n}{c} \right)$$

Since c and d are constants:

$$O(\log n)$$

11 Example: Multiplication by 3

Consider:

```
for(int i = 1; i < n; i = i * 3)
{
    // statement
}
```

Values:

$$1, 3, 9, 27, 81, \dots$$

After k iterations:

$$i = 3^k$$

Loop condition:

$$3^k < n$$

Taking logarithm:

$$k < \log_3 n$$

Therefore:

$$O(\log_3 n)$$

Since the base of logarithm is a constant:

$$O(\log n)$$

12 Example: Multiplication by 4

Consider:

```
for(int i = 1; i < n; i = i * 4)
{
    // statement
}
```

Values:

$$1, 4, 16, 64, 256, \dots$$

After k iterations:

$$i = 4^k$$

Condition:

$$4^k < n$$

Taking logarithm:

$$k < \log_4 n$$

Therefore:

$$O(\log_4 n)$$

or simply:

$$O(\log n)$$

13 Why Is It Called Logarithmic?

Suppose:

$$n = 16$$

For a linear loop:

$$0, 1, 2, 3, \dots, 15$$

There are approximately:

$$16$$

iterations.

For a multiplicative loop:

$$1, 2, 4, 8, 16$$

There are only:

$$\log_2 16 = 4$$

doubling steps.

For a larger value:

$$n = 1024$$

A linear loop requires approximately:

$$1024$$

iterations.

But a doubling loop requires:

$$\log_2 1024 = 10$$

iterations.

Therefore, logarithmic loops grow much more slowly than linear loops.

14 Easy Method to Solve Multiplicative Loops

For a loop of the form:

```
for(int i = c; i < n; i = i * d)
{
    // statement
}
```

follow these steps.

Step 1: Write the values

Write the first few values of i :

$$c, cd, cd^2, cd^3, \dots$$

Step 2: Find the value after k iterations

$$i = cd^k$$

Step 3: Apply the loop condition

$$cd^k < n$$

Step 4: Take logarithm

$$k < \log_d \left(\frac{n}{c} \right)$$

Step 5: Write Big-O

If c and d are constants:

$$\boxed{O(\log n)}$$

15 Important Special Cases

Loop	Time Complexity
$i = i * 2$	$O(\log n)$
$i = i * 3$	$O(\log n)$
$i = i * 4$	$O(\log n)$
$i = i * 10$	$O(\log n)$

The base of the logarithm does not affect Big-O when the base is a constant greater than 1.

16 Important Warning

Not every loop containing multiplication is automatically logarithmic.

For example:

```
for(int i = 0; i < n; i = i * 2)
```

is an infinite loop because:

$$0 \times 2 = 0$$

Similarly:

```
for(int i = -1; i < n; i = i * 2)
```

keeps moving in the negative direction:

$$-1, -2, -4, -8, \dots$$

Therefore, always check whether the loop variable is moving toward the termination condition.

17 Quick Revision

Pattern	Complexity
$i = i + 1$	$O(n)$
$i = i + 5$	$O(n)$
$i = i - 2$	$O(n)$
$i = i * 2$	$O(\log n)$
$i = i * 3$	$O(\log n)$
$i = i * 4$	$O(\log n)$
$i = 0, i = i * 2$	Infinite loop
$i = -1, i = i * 2$	Infinite loop

18 Student Shortcut

Remember these two patterns:

Pattern 1: Addition/Subtraction

If:

$$i = i + c$$

or

$$i = i - c$$

where c is a constant, then:

$$\boxed{O(n)}$$

Pattern 2: Multiplication/Division

If:

$$i = i * c$$

or

$$i = i / c$$

where $c > 1$ is a constant and the loop moves toward termination, then:

$$O(\log n)$$

Therefore:

Add/Subtract constant $\Rightarrow O(n)$ Multiply/Divide by constant $\Rightarrow O(\log n)$

19 Final Summary

1. If the loop variable increases by a constant amount, the loop is generally linear:

$$O(n)$$

2. If the loop variable is multiplied by a constant greater than 1, the loop is generally logarithmic:

$$O(\log n)$$

3. For:

$$i = i * 2$$

the values are:

$$1, 2, 4, 8, 16, \dots$$

4. After k iterations:

$$i = 2^k$$

5. Therefore:

$$2^k < n$$

6. Taking logarithm:

$$k < \log_2 n$$

7. Hence:

$$O(\log n)$$

8. Always check the starting value. Starting with 0 and repeatedly multiplying by 2 produces an infinite loop.

Final Rule: Multiplication by a constant gives logarithmic growth when the loop moves
--